Name: Kiruthika M
Reg No.: 192421239

ASSIGNMENT 1

CO1: Object-Oriented Programming with C++

Design, Implement and Validate the Fare Logic of a City Metro Smart-Card System

TASK 1 – Objects and OOP Analysis

Objects: SmartCard stores card number/balance and controls travel and deduction. Passenger stores passenger type

such as normal, student or senior. Journey stores zone, time and fare. Zone identifies Inner/Middle/Outer. FareTable

stores base fares and rules.

Procedural style: global variables + free functions such as calculateFare(), canTravel() and deduct(). In OOP, related

data and functions are grouped into classes, so the SmartCard can protect its balance.

Encapsulation: SmartCard balance is private. Abstraction: the user calls canTravel()/deduct() without knowing the

internal calculation. Inheritance: Passenger types could derive from Passenger, if needed. Polymorphism: different

passenger classes could override concession calculation.

Rules: fare must be non-negative; valid zone/time required; after payment balance must be at least Rs 50; peak is

08:00–09:59 and 17:00–19:59; money is stored in paise.

Complexity: one journey is O(1) time and O(1) extra space in both procedural and OOP versions. OOP mainly

improves maintainability and protects rules.

TASK 2 – Tokens, Constants and enum class

Five token types: keywords (class, int, if), identifiers (balance, fare), constants (5000, 20), operators (+, -, ==),

punctuators ({ }, ;, ( )).

Constants: #define is a preprocessor macro; const creates a typed constant; constexpr makes a value compile-time

constant when possible. constexpr is preferred here because it is typed, scoped and checked by the compiler.

```
enum class is safer than a plain enum because its values are scoped and do not implicitly convert to int.
enum class Zone { Inner, Middle, Outer };
constexpr long long INNER_PAISE = 1000;
constexpr long long MIDDLE_PAISE = 2000;
constexpr long long OUTER_PAISE = 3000;
constexpr int PEAK_SURCHARGE_PERCENT = 20;
constexpr long long MIN_BALANCE_PAISE = 5000;
```

TASK 3 – Data Types

Value

Type

Reason

Fare/balance

```
long long
```

Exact integer paise; avoids floating-point rounding

Zone

```
enum class
```

Limited named choices

Peak

```
bool
```

True/false

Hour

int

0–23

Journey count

```
long long
```

Safe for large counts

Character

char

Single character

Fractional calculation

```
double
```

Only where fractional values are genuinely required

Why paise? Rs 36.00 is exactly 3600 paise. With double, decimal values such as 0.1 may not be represented exactly

in binary, so repeated additions can produce small errors. For 3,000,000 journeys, long long is the safe accumulator.

Typical sizes are compiler/platform dependent, so run sizeof(...) on your own compiler and report those actual values. Use such as

INT_MAX and LLONG_MAX for your machine.

TASK 4 – Implicit Conversions + Predict Output

Predicted output:

3 3.5

C 68

1 true

s >= u

Explanation: a/b is integer division (7/2 = 3), then 3 is converted to double. static_cast(a)/b performs floating-point

division, giving 3.5. 'A'+2 is integer arithmetic, then the result 67 is stored in char as 'C'; ch+1 promotes C to int, giving

68. bool flag=5 converts nonzero 5 to true; default output is 1, boolalpha prints true. In s < u, -1 is converted to

unsigned int, producing a very large unsigned value, so the comparison is false.

Safer forms include double d{static_cast(a) / b}; and int x{3};. Brace initialization rejects narrowing such as int x{3.7};.

TASK 5 – Operators and Precedence

Peak rule: (hour >= 8 && hour < 10) || (hour >= 17 && hour < 20). Surcharge: fare += fare * 20 / 100. Concession can

be applied with a clearly parenthesized expression. The && operators are evaluated before ||, but parentheses improve

readability.

Predicted output:

11 16

10 6 4

6 6

3 1 -3 -1

18

Reason: 5+3*2 = 11, while (5+3)*2 = 16. x++ uses old x=5 and then x becomes 6; ++y changes y from 3 to 4 before

use, so z=5+4=9 — therefore the correct second line is 9 6 4. The final output is:

11 16

9 6 4

6 2

3 1 -3 -1

18

For x%4*2, / and % have the same precedence and are left associative: (x%4)*2 = 2. For negative integers, C++

integer division truncates toward zero: -10/3 = -3 and -10%3 = -1. The loop uses i = 0,3,6,9, so k=18. i=i++ should not

be used; it is a bad/undefined-style expression to avoid in practical code.

TASK 6 – Control Structures + Complete Code

Pseudocode: Read zone and hour → validate → determine peak → select base fare → add 20% surcharge using

integer paise → check balance – fare >= 5000 → deduct if allowed → print fare and balance.

```
#include &lt;iomanip&gt;
#include &lt;iostream&gt;
enum class Zone { Inner, Middle, Outer };
constexpr int BASE_PAISE[] = {1000, 2000, 3000};
constexpr int PEAK_SURCHARGE_PERCENT = 20;
constexpr long long MIN_BALANCE_PAISE = 5000;
long long fare(Zone z, bool peak) {
long long base = 0;
switch (z) {
case Zone::Inner: base = BASE_PAISE[0]; break;
case Zone::Middle: base = BASE_PAISE[1]; break;
case Zone::Outer: base = BASE_PAISE[2]; break;
}
if (peak) base += base * PEAK_SURCHARGE_PERCENT / 100;
return base;
}
bool canTravel(long long balance, long long f) {
return balance - f &gt;= MIN_BALANCE_PAISE;
}
int main() {
long long balance = 10000;
```

struct Trip { Zone zone; bool peak; };

Trip trips[] = {{Zone::Inner,false},{Zone::Outer,true},{Zone::Middle,false}};

```
for (const Trip&amp; t : trips) {
long long f = fare(t.zone,t.peak);
if (canTravel(balance,f)) {
```

balance -= f;

```
std::cout &lt;&lt; "Fare Rs " &lt;&lt; f/100 &lt;&lt; '.' &lt;&lt; std::setw(2)
```

<< std::setfill('0') << f%100 << std::setfill(' ');

```
} else {
std::cout &lt;&lt; "Denied (low balance)";


}
std::cout &lt;&lt; " balance Rs " &lt;&lt; balance/100 &lt;&lt; '\n';
}
}
```

Expected output: Fare Rs 10.00 balance Rs 90; Fare Rs 36.00 balance Rs 54; Denied (low balance) balance Rs 54. At

exactly 10:00 and exactly 20:00, the rule is off-peak because the upper bounds are exclusive.

TASK 7 – Procedural vs OOP

Procedural version:

```
long long balance = 10000;
bool canTravel(long long f) { return balance - f &gt;= 5000; }
void deduct(long long f) { if (canTravel(f)) balance -= f; }
```

OOP version:

```
class SmartCard {
private:
```

```
long long balance;
public:
explicit SmartCard(long long b) : balance(b) {}
bool canTravel(long long fare) const {
return balance - fare &gt;= 5000;
}
bool deduct(long long fare) {
if (!canTravel(fare)) return false;
```

balance -= fare;

```
return true;
}
long long getBalance() const { return balance; }
};
```

The private balance prevents other code from directly assigning an invalid value. Time and space per operation remain

O(1). OOP is easier to change because the minimum-balance rule is centralized.

TASK 8 – Safe Input and Output

```
int hour;
while (!(std::cin &gt;&gt; hour)) {
std::cin.clear();
std::cin.ignore(std::numeric_limits&lt;std::streamsize&gt;::max(), '\n');
std::cout &lt;&lt; "Invalid hour. Enter a number: ";
}
std::cout &lt;&lt; "Rs " &lt;&lt; std::fixed &lt;&lt; std::setprecision(2) &lt;&lt; rupees;
```

Use for numeric_limits. setw(2) and setfill('0') are useful for paise formatting. If using getline()+stoi(), catch

```
std::invalid_argument and std::out_of_range; getline processes a line of length L in O(L).
```

TASK 9 – Implementation and Test Table

Test

Expected

Inner, off-peak

Rs 10.00

Middle, off-peak

Rs 20.00

Outer, off-peak

Rs 30.00

Inner, peak

Rs 12.00

Middle, peak

Rs 24.00

Outer, peak

Rs 36.00

08:00

Peak

10:00

Off-peak

17:00

Peak

20:00

Off-peak

Balance leaving exactly Rs 50

Allowed

Balance leaving Rs 49.99

Denied

Unknown zone

Reject input

Negative recharge

Reject input

Letters for hour

Recover with cin.clear/ignore

TASK 10 – Expected vs Execution

Use the following formulas and replace the totals with your measured results. If every journey costs Rs 30, then 20,000

journeys = Rs 600,000; 500,000 = Rs 15,000,000; 3,000,000 = Rs 90,000,000. These values fit inside a 32-bit signed

```
int because they are below 2,147,483,647. However, larger totals or different fares can overflow int, so long long is
```

safer network accumulator. Double can represent these particular whole-rupee totals exactly, but decimal fractions can

accumulate rounding error.

Compile with g++ -std=c++17 -Wall -Wextra. Record the actual warnings from your compiler. A common warning in

Task 4 is signed/unsigned comparison.

TASK 11 – Final Recommendation

Recommended design: enum class Zone; constexpr fare constants; long long paise for money; bool for peak status;

SmartCard with private balance; FareTable/fare function for calculation; explicit validation; integer arithmetic for

surcharge; tests for every boundary. A fare revision should change the centralized fare table/constants rather than

repeated literals. Limitation: compiled-in fares require recompilation/redeployment for each revision.

For the whole network, O(1) fare calculation per journey and O(1) card state are suitable. Network totals should use

```
long long.
```

TASK 12 – DAY-4 DESIGN CHALLENGE

Changed requirements: Airport zone Rs 60; 10% weekend discount that does not combine with peak surcharge;

monthly pass for unlimited journeys in two chosen zones.

A. Impact: Add Airport to Zone. Add 6000 paise to fare data. Add weekend rule. The peak/discount expression must

make the rules mutually exclusive. Add a Pass class with two allowed zones and active status.

B. Evaluation: The existing switch can absorb Airport by adding a case, but repeated switch logic becomes harder to

maintain as zones grow. SmartCard still works because its responsibility—balance and travel permission—does not

depend on the exact number of zones.

C. Recommended adaptation: Use a fare table indexed by Zone and separate the discount calculation. Use Pass to

represent unlimited travel in two zones.

```
enum class Zone { Inner, Middle, Outer, Airport };
constexpr long long BASE[] = {1000, 2000, 3000, 6000};
long long calculateFare(Zone z, bool peak, bool weekend) {
long long f = BASE[static_cast&lt;int&gt;(z)];
if (weekend) {
```

f = f * 90 / 100;

// weekend discount

```
} else if (peak) {
```

f += f * 20 / 100;

// peak only when not weekend

```
}
return f;
}
class Pass {
```

Zone a, b;

```
bool active;
public:
Pass(Zone x, Zone y) : a(x), b(y), active(true) {}
bool covers(Zone z) const {
return active &amp;&amp; (z == a || z == b);
}
};
```

D. Defence: The table reduces duplicated fare logic and makes adding Airport straightforward. Separate weekend/peak rules prevent accidental combination. Pass isolates the new monthly-pass feature. Test Airport off-peak, Airport peak, weekend Airport, weekend+peak, and trips covered/not covered by the pass.